

Applying **TRANSFORMATICS** in GENETICS: The Mathematics of Genome Sequencing

Willrich J. Lutalo*

joewillrich@gmail.com, jwl@nuchwezi.com

February 18, 2026

Abstract

This work is part of an upcoming book(via Taylor and Francis), “Applying TRANSFORMATICS in GENETICS” by the author. Sequencing is such a central concept in genetics that it surely warrants its own chapter in a book on any modern mathematics of genetics such as is being developed and being formalized in this book. Moreover, it is not just in genetics that the idea of sequencing finds immediate relevance, just as with many of the other concepts being treated of in this book, much of what mechanics and theory is being developed about sequence processing, analysis and in general, the algebra of sequences, shall find many useful and sometimes fundamental applications in domains such as computer science, physics, pure and applied mathematics, medicine, economics, etc. In this chapter though, focus is on the problem of *genome sequencing*, with special attention being given to identifying the core problems underlying such sequencing as it is encountered in contemporary genetics literature and practice, and then, leveraging the new mathematics of transformatics[1], to develop entirely new formalisms and theory concerning how such sequencing might be accomplished, but also how it might be applied. We cover the matter of sequence alignment, with special, deep treatment of the issue of computing the maximum common subsequence from any collection of disparate sequences. We then handle the issue of computing a consensus sequence given any two sequences well aligned and with potential disparities in the overlapping sections (specifically for genetic sequences here), then finally apply these two solutions to the issue of formally specifying the problem of genome sequencing, to which we provide a solution in form of a special sequence transformer that we call the genome sequencer. Finally, we see how the theory thus developed might be used to implement such critical applications such as DNA or genetic tests, and how these ideas culminate in a law about the unique identity of any entity for which there exists some kind of genome sequence.

Keywords: Modern Mathematics, Transformatics, Sequence Algebra, Sequence Analysis, Sequence Processors, DNA, Genome Sequencing, Sequence Transformers.

1 Relevant Background

It is important that we have some general appreciation of the core concept about which this chapter¹ is dedicated; *sequencing*. For this matter, we shall want to bring to mind one of the popular definitions of the term as employed in computer science for example:

Sequencing The procedure by which ordered units of data (octets or messages) are numbered, transmitted over a communications network (which may rearrange their order), and reassembled into the original order at their destination.

— Oxford Dictionary of Computing[3]

That is the closest concept or definition I could glean from my collection of academic literature and books in my private library at Nuchwezi Research Lab, however, this other definition, obtained via Microsoft’s Copilot might also help shed more light on the particular concept of concern here: *genome sequencing*:

Genome Sequencing²[4]: Genome sequencing is the process of determining the complete DNA (or RNA) sequence of an organism’s genome. It identifies the precise order of nucleotide bases (A, T, C, G) across all or most of the genetic material, providing a comprehensive "blueprint" of the organism’s hereditary information. This technique is fundamental in biology, medicine, and biotechnology, enabling insights into genetic variation, disease mechanisms, and evolutionary relationships.

— Microsoft Copilot[5]

*Currently a volunteering & Independent Principal Investigator at Nuchwezi Research — <https://nuchwezi.com>

¹Part of new material in upcoming new edition of earlier book draft[2].

²Also see: https://en.wikipedia.org/wiki/Whole_genome_sequencing

1.1 Problems and Knowledge Gaps

First, we shall mostly assume there is essentially no reliable mathematical formalisms or theory concerning the important matter of genome sequencing³. And so, shall set out to construct and develop such a formalism ourselves, from what we already know.

Second, from the theoretical and exploratory reviews we have conducted leading up to this moment, and what some authorities[6] say about the subject, we know that genome sequencing as a practice leveraging computational and mathematical principles and concepts, or rather, when approached from a predominantly mathematical theory perspective, still has some ambiguities and unresolved problems that we have identified and intend to iron out especially in this chapter.

These for example include an overall lack of proper mathematical formalisms around issues such as:

1. How [genome] sequencing works or what it is exactly; how can it be described computationally?
2. If we treat of genome sequencing using the language of transformatics, and think of it in terms of processing some collection of sequences to arrive at some resultant genome sequence, how should one arrive at such a resultant sequence then?
3. What is a “consensus sequence”[6]?
4. What is meant by “a most representative genome sequence” or perhaps an identifying **na-Sequence** given some living organism or particular aspects of it?
5. Could we (perhaps conceptually, or theoretically) talk of sequencing a population instead of an individual organism, an entire species or perhaps (sic) non-living things too? How?
6. Could we pin down a genome sequence or some na-Sequence responsible for say the expression⁴ of some particular protein, some genetic mutation, a behavioral trait, anatomical or psychological anomalies and pathologies, the material basis of particular or general diseases, incapacities, etc. using a mathematical theory or language? How?

Seeing as the matter of determining the actual na-Sequence that correctly identifies a person or thing might be hinging on the case of attempting to reconstruct a book by mathematically piecing together disparate pieces of it — some in alignment, some out of alignment, some, supersets of others, some prefixes or suffixes of others, etc.⁵, we then start to come to some appreciation of how best we might approach the problem’s solution.

Also, though we might not immediately surface such philosophical speculations here, the difficulties and problems thus raised above, might likewise apply to or at least hint at the challenges there might be, in attempting to think of or practically resolve the matter of “computationally sequencing anything” — the idea being, to not limit the concept of *sequencing* to only living things or living organisms only, but to also think generally and abstractly **concerning sequencing anything expressible in consensus reality via some kind of specification sequence**⁶.

Thus, we might want to also think of, keep in mind, the possibility of sequencing the peculiar, miniscule aspects of consensus reality — sub-atomic particles that make up atoms that make up molecules that make up living things on the miniature scale, and then all the way up to planets, stars and galaxies on the grand, cosmic and universal scale. One might wonder, and correctly so, is such sequencing possible? Does it/would it make sense? Is it necessary or even useful? These, and related problems are among the concerns we shall attempt to address in the rest of this chapter, but also, bits of what we can not immediately answer, perhaps we shall tackle in later chapters or future works⁷.

³For those that do not know, a formal definition is presented in **Definition 2**.

⁴Related, but not to be treated of in this chapter but instead in **Chapter 10**[2], is the matter of mathematically approaching how the expression — visual, material or spatial, of any meaningful *genome* sequence — could be for a basic organism, an aspect of it, or even the case of *anomalous entities* such as viruses, mere proteins or such, could be approached.

⁵This analogy shouldn’t come as a surprise, when compared to how we found it best to describe DNA using a figurative metaphor (see **Introduction**), but also based on how some authorities seem to be teaching this subject to their students[6].

⁶Or perhaps an “identifying sequence”.

⁷**FUTURE WORK:** Explore, philosophically, but also mathematically, whether or not it makes sense to think of the [genome] sequencing of arbitrary things; not necessarily only living/biological entities, but also any coherent material entities such as particular atoms, molecules,... all the way to entire ecosystems, planets and star-systems perhaps! For example, given that the manifestation of biological lifeforms on some particular planets within a star system heavily relies on some initial conditions inherent in the early formation of the star [system] itself, could it be that something fundamental in matter could underlie all manifestation or expression of existence other than just biological life, and that we might somehow be able to pin it down to some kind of identifier sequence? A genome for an entire planet or star system?

2 An Original Treatment: 3 Core Problems to Solve

We shall develop our theory in bits, starting with the most basic, most fundamental, and then basing on that, to construct higher aspects of the necessary mathematical structures and system.

The method is simple; we have distilled 3 core problems whose solutions would culminate in the formal and mathematical definition of genome sequencing. The 3 problems are

1. **PROBLEM 1:** Computing the best or rather correct common subsequence given any two or more potentially dissimilar sequences.
2. **PROBLEM 2:** Computing the correct sequence that resolves conflicts and correctly expresses variations at each particular position (essentially a column) in the alignment matrix depicting a definite side-by-side alignment of two or more sequences with an overlap in the alignment position being considered.
3. **PROBLEM 3:** Computing the Genome Sequence.

We shall tackle each of these problems in its own section, one by one, until we arrive at the original objective of this chapter, in **Section 3**.

2.1 PROBLEM G1: Computing Maximum Common Subsequence: $\Theta_1 \diamond \Theta_2$

Problem 1 (*Computing MCS:*). : $\Theta_1 \diamond \Theta_2$] Given sequences $\Theta_1 = \langle k, l, m, o, p, x, y, z \rangle$ and $\Theta_2 = \langle b, c, a, y, z, k, l, m, o, p \rangle$, find the longest common subsequence they share — also to be denoted $\Theta_1 \diamond \Theta_2$, their **Maximum Common Subsequence**.

Solution 1. We shall specify a transformer, $tMCS(\Theta_1, \Theta_2)$ that produces the required solution from the input sequences as such:

Transformer 1 (The **tMCS** Transformer). $\langle \Theta_1, \Theta_2 \rangle \xrightarrow{O_{tMCS}(\Theta_1, \Theta_2)} \Theta_1 \diamond \Theta_2$;
 $\Theta_1 \diamond \Theta_2 = \Theta_2 \diamond \Theta_1 \quad \wedge \quad 0 \leq \nu(\Theta_1 \diamond \Theta_2) \leq \text{Max}(\nu(\Theta_1), \nu(\Theta_2))$
 $\wedge \quad \forall \omega \in \Theta_1 \diamond \Theta_2 \quad \exists \omega \in \Theta_1 \wedge \exists \omega \in \Theta_2$
 $\wedge \quad \forall \omega_i, \omega_j \in \Theta_1 \diamond \Theta_2 : I(\omega_i, \Theta_1 \diamond \Theta_2) < I(\omega_j, \Theta_1 \diamond \Theta_2)$
 $\implies \quad I(\omega_i, \Theta_1) < I(\omega_j, \Theta_1) \quad \wedge \quad I(\omega_i, \Theta_2) < I(\omega_j, \Theta_2)$
 $\wedge \quad \forall \Theta_1 \diamond \Theta_2, \quad \Theta_1 \diamond \Theta_2 \subset \Theta_1 \quad \wedge \quad \Theta_1 \diamond \Theta_2 \subset \Theta_2$

□

Solution 1 is easier said than done. However, we shall attempt to formally and rigorously express it by formalizing what exactly the $tMCS(\Theta_1, \Theta_2)$ transformer does when expressed as a computable algorithm. Basically, we are to compute the MCS via **Algorithm 1** specified hereafter.

Algorithm 1 (The **tMCS** Algorithm⁸).

1. **INITIALIZE** the global maximum common sequence, O_g^m , to the empty sequence⁹.
2. **GIVEN** a collection of n sequences, $\Theta^n = \{\Theta_1, \Theta_2, \dots, \Theta_n\}$.
3. **START** by computing the cardinality of each sequence, $\Theta_i \in \Theta^n$, and note the **largest sequence cardinality**, $\text{Max}(\nu(\Theta_i) \quad \forall \Theta_i \in \Theta^n) = \text{Max}(\nu(\Theta_i))$, for the longest sequence, Θ_m such that $\text{Max}(\nu(\Theta_i)) = \nu(\Theta_m)$.
4. **COMPUTE** the cardinality of a sequence that can exactly contain the given sequences each concatenated to the other, but with the longest sequence duplicated. Essentially, compute:

$$C_R = \nu(\Theta_m) + \sum_{\forall \Theta_i \in \Theta^n} \nu(\Theta_i) \quad (1)$$

⁸**IMPORTANT:** Note that, even though the MCS transformer, **Transformer 1**, is defined for just two sequences, and yet, the tMCS algorithm based off of it is designed to handle any arbitrary number of sequences and not just a pair.

⁹Refer to **Foundation Idea 1**[1].

5. ***SORT*** the collection of sequences, Θ^n , in descending order¹⁰ of the sequence cardinality. Let the sorted sequence collection be denoted $\hat{\Theta}^n = \langle \Theta_m, \Theta_{2*}, \dots, \Theta_{n*} \rangle$ — where the first item in $\hat{\Theta}^n$, Θ_{1*} is essentially Θ_m .
6. ***SELECT*** the first two sequences from $\hat{\Theta}^n$, such as Θ_m and Θ_{2*} — or rather/generally, $\hat{\Theta}^n[1]$ and $\hat{\Theta}^n[2]$, as the two sequences we shall use to search for and derive the potential MCS.
7. ***POSITION*** the two sequences, Θ_m and Θ_{2*} within two adjacent arrays $A_1^{C_R}$ and $A_2^{C_R}$ of equal length C_R — for simplicity, we shall just denote them as A_1 and A_2 , in such a way that Θ_m occupies in its containing array A_1 , positions from m to $2m$, but for simplicity's sake¹¹, which we shall just treat as though 1 to m :

$$A_1 : [1, m]$$

While Θ_{2*} occupies positions in the range¹²:

$$A_2 : [(1 + I(\Theta_m[m], A_1), \quad I(\Theta_m[m], A_1) + \nu(\Theta_{2*})]$$

Or, equivalently, the range:

$$A_2 : [m + 1, \quad m + \nu(\Theta_{2*})].$$

Which we can also express visually as:

A_1	...	$a_{1,1}$	$a_{1,2}$	...	$a_{1,m}$					...
A_2	...					$a_{2,1}$	$a_{2,2}$	...	$a_{2,\nu(\Theta_{2*})}$	...

8. ***UPDATE*** the two sequence alignment arrays as such:

- (a) ***KEEP*** the first array, A_1 , unchanged (since it contains the longest sequence, $\Theta_{1*} \in \hat{\Theta}^n$).
- (b) ***SET*** $j = z$, where z is the ***shift step size*** — simplest scenario, $z = 1$.
- (c) ***SHIFT*** each element in the second sequence by j -steps to an earlier position in array A_2 , so that the two sequences Θ_m and Θ_{2*} have an overlap across the alignment arrays/matrix in positions spanning range:

$$A_1 : [m - j, \quad m]$$

and so that Θ_2 shall be occupying the updated range:

$$A_2 : [m + 1 - j, \quad m + \nu(\Theta_{2*}) - j]$$

¹⁰**FUTURE WORK:** We especially prefer descending order by sequence cardinality because, as we shall come to appreciate in the rest of the tMCS algorithm, it is more likely that the MCS is discovered or developed based on the longest common [sub-]sequence (in the longest sequences in the collection) than if we had started out with the shortest sequences. In fact, it also is somewhat very logical, since, the longest sequence shall most likely also contain elements and subsequences that shorter sequences have or are derived from than vice versa! Thus we can converge towards the MCS faster that way. Also, this is the *optimistic strategy* — believing that the most common subsequence shall most likely already exist in the longest sequences we are analyzing — which makes sense for genome sequencing given we are typically analyzing sequences from the same entity or individual or species, and that DNA or genetic code for the same individual or entity stays consistent across all or most of the samples from that entity. But otherwise the pessimist route might be to start out with the shortest sequences in the collection (esp. where we are dealing with totally antagonistic entities). But is this always optimal? Should it stay this way even when we are attempting [genome] sequencing for non-biological or non-genetic sequences?

¹¹We shall later see (in **Step#8(c)**) that it is useful to position Θ_m this way, because then, the second sequence's positions are all that need to be changed — items being shifted leftwards in A_2 relative to A_1 , while those of the longest sequence in A_1 remain fixed. But to keep the mathematical formalisms simple, we shall especially write our expressions as though elements of Θ_m were positioned from index 1 in A_1 — these details especially matter to programmers and software engineers that shall want to translate the algorithm into working computer program source-code for various different languages and type-systems.

¹²We write " $\Theta_{2*}[1]$ " in our formulation here, to mean the first element in Θ_{2*} — i.e. position 1 has value 1, and the n th position has value n , not usual (computer-science) $n - 1$

- (d) **SCAN** the overlapping sections in both sequences, in positions $[m-j, m]$ and by comparing each item in that range — i.e. $\Theta_m[i]$ Vs $\Theta_{2*}[i]$, determine the longest common subsequence, or rather the **MCS**, thus:

Assume the overlap is of length¹³ ϕ such that the overlap can be visually depicted as such:

$\approx \Theta_m$	$a_{1,m-\phi+0}$	$a_{1,m-\phi+1}$	$a_{1,m-\phi+2}$	$\dots$	$a_{1,m-\phi+j}$
$\approx \Theta_2$	$a_{2,m-\phi}$	$a_{2,m-\phi+1}$	$a_{2,m-\phi+2}$	$\dots$	$a_{2,m}$
$(\Theta_m[i] == \Theta_{2*}[i])?$	1	1	0	$\dots$	1
$\approx (\Theta_m[i] == \Theta_{2*}[i]) \approx O^\phi$	$f(\Theta_m[i], \Theta_{2*}[i])$	$a_{1,m-\phi+1}$	0	$\dots$	$f(a_{1,m}, a_{2,m})$

CREATE another array of maximum length ϕ , i.e. O^ϕ , which contains at most ϕ elements generated thus:

- $O^\phi[i]$ contains 0 if the corresponding two elements in Θ_m and Θ_{2*} didn't match. Otherwise it contains the matching element value from the first sequence. Essentially, resolve what goes into O^ϕ at each of the overlap indices — $i : i \in [m-\phi, m]$ — via the function:

$$f(\Theta_m[i], \Theta_{2*}[i]) = \begin{cases} \Theta_m[i] & \Theta_m[i] == \Theta_{2*}[i], \\ 0 & \Theta_m[i] \neq \Theta_{2*}[i] \end{cases} \quad (2)$$

- **TRUNCATE** O^ϕ to its longest non-zero/non-empty subsequence, and **SET** that as $\bar{O}^\phi$.
- **IFF** $\mathfrak{L}(\bar{O}^\phi) > \mathfrak{L}(O_g^m)$, then **UPDATE** or replace O_g^m with $\bar{O}^\phi$
 $\implies O_g^m := \bar{O}^\phi$

- (e) **UPDATE** $j \implies j := j + z$
(f) **IFF** $j = \mathfrak{L}(\Theta_m) + \mathfrak{L}(\Theta_{2*})$, then **GOTO**¹⁴ **Step#9**
(g) **ELSE ITERATE** from **Step#c**

9. **UPDATE** Θ_{2*} as such:

- **COMPUTE** index of next comparison sequence, k , as such:

$$k = I(\Theta_{2*}, \hat{\Theta}^n) + 1 \quad (3)$$

- **IFF** $k > n$, then **RETURN**¹⁵ O_g^m .
- **ELSE LET** $\Theta_{2*} := \hat{\Theta}^n[k]$

10. **IFF** $\mathfrak{L}(\Theta_{2*}) < \mathfrak{L}(O_g^m)$, then **RETURN**¹⁶ O_g^m .

11. **ELSE ITERATE** from **Step#7**

□

¹³Essentially, since the overlap spans the range $[m-j, m]$, then $\phi = m - (m-j) = j$

¹⁴At this juncture, we shall have essentially finished sliding the second sequence, Θ_{2*} , past all the elements of the first sequence so that at $j = \mathfrak{L}(\Theta_m) + \mathfrak{L}(\Theta_{2*})$ there is no more overlap possible, and so we can move on to the next sequence.

¹⁵Basically, we have reached the end of our sorted list of sequences and whatever we currently have as MCS must be the solution.

¹⁶Basically, once we find that the current MCS is longer than any of the remaining sequences in the collection, we might as well stop searching and return it as the final solution.

At which point, the MCS between these two sequences, $\Theta_m \diamond \Theta_{2*} = O^\phi = \langle y, z \rangle$, and that $\nu(O^\phi) = 2 < \nu(O_g^m)$, and so then we cannot update the global MCS so that, **still**, $O_g^m = \langle k, l, m, o, p \rangle$.

10. We finally stop when $j = \nu(\Theta_m) + \nu(\Theta_{2*}) = 10 + 8 = 18$, and at which point we shall find that we have not encountered any other overlap longer than our earlier MCS, and so that, given we also have no more sequences to compare against, we must **return the final MCS** as $O_g^m = \langle k, l, m, o, p \rangle$.

Thus, after applying **Algorithm 1** to our two-member sequence collection, we find that the MCS solution we arrive at is $\langle k, l, m, o, p \rangle$ for that particular higher-order sequence. We can confirm that the solution conforms to the specifications of **Transformer 1**, such as noting that:

- From $\langle \Theta_1, \Theta_2 \rangle$, we have $\Theta_1 \diamond \Theta_2 = \Theta_2 \diamond \Theta_1 = \langle k, l, m, o, p \rangle$.
- We see that $\nu(\Theta_1 \diamond \Theta_2) = 5$, so that $0 \leq \nu(\Theta_1 \diamond \Theta_2) \leq \text{Max}(10, 8)$
- We also see that all elements in $\Theta_1 \diamond \Theta_2$ also exist in either sequences.
- And also that the relative order of elements in $\Theta_1 \diamond \Theta_2$ preserves the relative order of corresponding symbols in both Θ_1 and Θ_2 .
- And that finally, we see that $\Theta_1 \diamond \Theta_2$ is a sub-sequence of both Θ_1 and Θ_2 .

2.2 PROBLEM G2: Computing Overlap Resultant Sequences: $\Theta_1 * \Theta_2$

Problem 2 (*Computing ORS: Overlap Resultant Sequence*). : $\Theta_1 * \Theta_2$] Given two overlapping/well-aligned sequences Θ_1 and Θ_2 of the same length but potentially different terms at any position in the range $[1, \nu(\Theta_1)]$, how to compute a resultant sequence $\Theta_1 * \Theta_2$ that is most meaningful for DNA sequences?

Solution 2. Assuming $\Theta_1 = \langle \prod_{i=1}^n a_i, \rangle$ and $\Theta_2 = \langle \prod_{i=1}^n b_i, \rangle$, construct Θ^* via the following *tORS* transformer:

Transformer 2 (The **tORS** Transformer).

$$\langle \langle \Theta_1 \langle \prod_{i=1}^n a_i, \rangle, \Theta_2 \langle \prod_{i=1}^n b_i, \rangle \rangle \xrightarrow{O_{tORS}(\Theta_1, \Theta_2)} \Theta^* = \Theta_1 * \Theta_2;$$

$$\nu(\Theta_1 * \Theta_2) = \nu(\Theta_1) = \nu(\Theta_2) = n$$

$$\wedge \Theta^* = \langle \prod_{i=1}^n f(a_i, b_i), \rangle$$

for $f(x, y)$ the **ORS resolver function** defined as such:

$$f(x, y) = \begin{cases} x, & x == y, \text{ terms are the same} \\ x, & x = \neg(y), \text{ one term is the complement of the other} \\ y & \text{otherwise.} \end{cases}$$

□

Concerning that succinct solution, note that we have covered sequence complements in **Chapter 5**[2] and especially for DNA or na-Complements in **Section 5.1**. We shall not delve into the intricacies of translating **Transformer 2** into a working generic algorithm such as we did for **Problem G1**, because we have already well illustrated how the process works — see or compare **Algorithm 1** to **Transformer 1**, and also see how we have applied both the algorithm and underlying transformer to some illustrative test-cases in **Section 2.1.1**. Thus, it should not be hard for the serious student, researcher or genetics engineer to apply, verify and/or extend the above proposed solution to their particular needs.

Finally, note that being able to compute the **ORS** is quite useful, as it underlies the computation of the consensus overlapping sequence from which a genome sequence is then derivable as we see in the next section.

2.3 PROBLEM G3: Computing the Genome Sequence (GS): $(\Theta_1 \diamond \Theta_2)^*$

Problem 3 (*Computing GS: Genome Sequence*). : $(\Theta_1 \diamond \Theta_2)^*$] Given any two sequences, Θ_1 and Θ_2 , that are the elements of a sequence collection, Θ^n , compute their **resultant genome sequence**, denoted $(\Theta_1 \diamond \Theta_2)^*$, that contains at core, their longest common subsequence (the MCS), $O^m = (\Theta_1 \diamond \Theta_2)$, potentially padded on either side by the specially computed prefix and suffix subsequences from either input sequence as such:

$$(\Theta_1 \diamond \Theta_2)^* = \Theta_1^a \cdot (\Theta_1^b * \Theta_2^b) \cdot (\Theta_1 \diamond \Theta_2) \cdot (\Theta_1^c * \Theta_2^c) \cdot \Theta_2^a \quad (6)$$

And where the special terms are derived and named as such:

- Θ_1^a is the non-overlapping section of Θ_1 before the **COS** — to be introduced soon.
- $(\Theta_1^b * \Theta_2^b)$ is a consensus sequence from Θ_1 and Θ_2 such that its elements are the result of applying the **ORS resolver function** to compute $f(\Theta_1[i], \Theta_2[i])$ as specified in **Transformer 2**. Thus, it is one of the overlapping resultant sequences (ORS) resulting from Θ_1 and Θ_2 — this particular one occurring **before the MCS**.
- $(\Theta_1 \diamond \Theta_2)$ is the **maximum common subsequence**, MCS, O_g^m , of Θ_1 and Θ_2 . It is the heart of the computed genome sequence.
- $(\Theta_1^c * \Theta_2^c)$ is the other ORS from Θ_1 and Θ_2 , this one occurring **after the MCS**.
- Θ_2^a is the non-overlapping section of Θ_2 after the **COS**.

COS, $\overline{\Theta_1 \Theta_2}$, which is the **Consensus Overlapping Sequence**, padded on either side by the non-overlapping sections of either Θ_1 or Θ_2 , has the following properties:

- $\overline{\Theta_1 \Theta_2} = (\Theta_1^b * \Theta_2^b) \cdot (\Theta_1 \diamond \Theta_2) \cdot (\Theta_1^c * \Theta_2^c)$
- $\Theta_1 \approx \Theta_1^a \cdot \overline{\Theta_1 \Theta_2}$
- $\Theta_2 \approx \overline{\Theta_1 \Theta_2} \cdot \Theta_2^a$

So that we could rewrite **Equation 6** as such:

$$(\Theta_1 \diamond \Theta_2)^* = \Theta_1^a \cdot \overline{\Theta_1 \Theta_2} \cdot \Theta_2^a \quad (7)$$

Finally, we know that the cardinality of the genome sequence, $\mathfrak{u}((\Theta_1 \diamond \Theta_2)^*)$, is bounded thus:

$$\mathfrak{u}(\Theta_m) \leq \mathfrak{u}((\Theta_1 \diamond \Theta_2)^*) \leq C_R - \mathfrak{u}(\Theta_m) \quad (8)$$

As per **Equation 1** and the sensibilities of GTNC[7], and which inequality, in its most general form (applicable to a genome sequence derived from more than just two sequences) we might also depict as¹⁷:

$$\text{Max}(\mathfrak{u}(\Theta_i)) \leq \mathfrak{u}(\diamond(\Theta^n)) \leq \sum_{\forall \Theta_i \in \Theta^n} \mathfrak{u}(\Theta_i) \quad (9)$$

¹⁷**NOTE:** we write $\diamond(\Theta^n)$ in **Equation 9** instead of $\Theta_1 \diamond \Theta_2$ to mean the MCS resulting from processing all (potentially more than just two) sequences in some collection or higher-order sequence Θ^n . And so that, even though it results from a higher-order sequence, $\diamond(\Theta^n)$, just like $\Theta_1 \diamond \Theta_2$, is a first-order (or rather, “flat”) sequence.

3 The Genome Sequencer

Definition 1 (A Genome Sequencer). A solution to **Problem 3**, is a special sequence processor, $\tilde{T}(\Theta^n)$, that can compute and return a resultant sequence, R_{Θ^n} , also known as the **genome sequence**, defined as^a

$$R_{\Theta^n} = (\Theta_m \diamond (\hat{\Theta}^n \setminus \Theta_m))^* \quad (10)$$

Such that Θ_m is the longest sequence in Θ^n — a collection of n sequences or $\hat{\Theta}^n$, its version with member sequences sorted in descending order of sequence cardinality, and that $\tilde{T}(\Theta^n)$ produces/generates/computes R_{Θ^n} by iteratively computing and updating the genome sequence in $n-1$ iterations over $\hat{\Theta}^n$ as such

$$\begin{aligned} \textbf{Transformation 1. } R_{\Theta^n}^0 &\xrightarrow{O_{tGS}(R_{\Theta^n}^0, \hat{\Theta}^{n-1}, 1)} R_{\Theta^n}^1 \xrightarrow{O_{tGS}(R_{\Theta^n}^1, \hat{\Theta}^{n-2}, 2)} R_{\Theta^n}^2 \xrightarrow{O_{tGS}(R_{\Theta^n}^2, \hat{\Theta}^{n-3}, 3)} R_{\Theta^n}^3 \\ \dots &\xrightarrow{O_{tGS}(R_{\Theta^n}^{j-1}, \hat{\Theta}^{n-j}, j)} R_{\Theta^n}^{j+1} \dots \xrightarrow{O_{tGS}(R_{\Theta^n}^{n-2}, \hat{\Theta}^1, n-1)} R_{\Theta^n}^n \end{aligned}$$

To produce the final genome sequence, $R_{\Theta^n}^n$ via the **tGS Transformer** defined as such:

Transformer 3 (The tGS Transformer, $\tilde{T}(\Theta^n)$).

$$\begin{aligned} R_{\Theta^n}^j &\xrightarrow{O_{tGS}(R_{\Theta^n}^{j-1}, \hat{\Theta}^{n-j}, j)} R_{\Theta^n}^{j+1}; \\ R_{\Theta^n}^0 &= \Theta_m \quad \wedge \quad R_{\Theta^n}^{j+1} = (R_{\Theta^n}^j \diamond \hat{\Theta}^{n-j}[1])^* \quad \forall j \geq 1 \\ \square \end{aligned}$$

and $R_{\Theta^n}^{j+1}$ is the resultant genome sequence after processing $\hat{\Theta}^n$ with the first j sequences removed from it, and with the previous genome sequence, $R_{\Theta^n}^j$ passed to it, as the current best candidate genome sequence so as to produce that next genome sequence $R_{\Theta^n}^{j+1}$. Also, for a sequence collection of n member sequences, we write R_{Θ^n} to also mean $R_{\Theta^n}^n$ as in **Equation 10**.

^aNote that in our formalism here we express the **sequence difference** of $\{\Theta_m\}$ from Θ^n as $\Theta^n \setminus \Theta_m$ — essentially using a set-theoretic notation even though we are actually dealing with sequences here. This, so as to keep the notation simple, but otherwise, later also appeal to the same concept when considering the remaining higher-order sequence after the first j member sequences have been processed and removed from it, with the notation Θ^{n-j} .

Thus, **Transformer 3**, $\tilde{T}(\Theta^n)$, is a genome sequencer, and is essentially a transformer that reduces a matrix of na-Sequences to a resultant sequence, R_{Θ^n} , that is the resultant genome sequence of a specie, population, or any natural entity¹⁸ represented by or expressed by the [potentially incomplete or approximate] genome sequences in the collection Θ^n .

It shall be worth noting that, for the special case where we merely have a collection of just two sequences from which to compute the genome — such as when $\Theta^n = \langle \Theta_1, \Theta_2 \rangle$ for some two [na-]sequences, Θ_1 and Θ_2 , we shall readily find that applying **Transformer 3** to that pair of sequences reduces to the case we have encountered in **Problem 3**, and that then, we find that:

$$R_{\Theta^n} = (\Theta_m \diamond (\hat{\Theta}^n \setminus \Theta_m))^* = (\Theta_1 \diamond \Theta_2)^* = \Theta_1^a \cdot \overline{\Theta_1 \Theta_2} \cdot \Theta_2^a \quad (11)$$

Finally, it shall help to put it out clearly, what our working definition of **Genome Sequencing** is, given what we now know.

¹⁸In principle though, it is not that we can not apply this theory to the computing of genome sequences for artificial or rather conceptual entities too.

Definition 2 (Genome Sequencing). When a collection of *na-Sequences*, Θ^n , is reduced to a single *na-Sequence*, Θ_{gs} , via a process that correctly aligns all the sequences in Θ^n such that the resultant Θ_{gs} is the best representative consensus sequence given all the variations in Θ^n , we call Θ_{gs} the **genome sequence** of Θ^n , and we know that if some other sequence, Ω_{na} is considered to be a legitimate genome sequence of Θ^n , then it must be equivalent to Θ_{gs} , where Θ_{gs} can be computationally produced from Θ^n , via **Transformer 3** as such:

Transformation 2. $\Theta^n \xrightarrow{\tilde{T}(\Theta^n)} \Theta_{gs}$

and equivalently:

$$\Theta_{gs} = \tilde{T}(\Theta^n) = R_{\Theta^n} \quad (12)$$

We call the process of reducing Θ^n to R_{Θ^n} , **genome sequencing**, and can use it to test or prove if indeed some sequence Ω_{na} is a legitimate genome sequence of Θ^n — essentially, testing if the two sequences belong to the same entity or perhaps species, by running the test:

$$0 \leq \tilde{A}(R_{\Theta^n} \rightarrow \Omega_{na}) \leq \epsilon \quad \vee \quad 0 \leq \tilde{A}(\Theta_{gs} \rightarrow \Omega_{na}) \leq \epsilon \quad \implies \quad \Omega_{na} \cong \Theta_{gs} \quad (13)$$

Otherwise $\Omega_{na} \neq \Theta_{gs}$, for $\epsilon \leq 1$.

4 The Identity Genome Sequence (IGS) Law

Law 1 (The Identity Genome Sequence Law). The Identity Genome Sequence, $R_{\Theta^n}(\Omega) : \mathbb{N} \times \psi_{DNA}$, is derived from some collection of sequences, $\Theta^n \langle \prod_{i=1}^n \theta_i \rangle$ such that $\theta_i \subset \Omega \vee \theta_i \approx \Omega$ are approximations of Ω , the longest known genome sequence of the entity, whose exact genome sequence would be Ω , and which can uniquely identify it. For sufficiently large n , and where the sample sequences θ_i were read or generated correctly by scanning or sequencing the entity, then

$$R_{\Theta^n}(\Omega) \approx \Omega_m \quad (14)$$

Where Ω_m is the known best approximation of the genome sequence that correctly identifies any instance of a member of the kind Ω .

NOTE:

On Consequences of IGS

One potentially significant consequence of **Law 1** is that every distinct living thing — unique or distinct animal, plant, eukaryote or prokaryote, has a distinct identifying genome sequence, Ω , that we can discover and approximate via significantly many readings of particular or representative [sub-]sequences of the DNA of the observable expressions of Ω .

However, given that in reality it might not be exactly possible to exhaustively compute the correct value of R_{Θ^n} , nor tell exactly when that approximation approaches Ω exactly, it makes one wonder whether we can ever accurately know what it is that exactly underlies the expression of our reality — living things or not^a.

^aFirst, because, for currently known living things, the sheer complexity of correctly or accurately sequencing a genome for even the simplest organisms is constrained by the sheer amount of large resources required to correctly process millions to billions of nucleotides, and that typically, it is not possible to obtain an entire genome sequence in a single reading. And then, note that there is the queer possibility that a kind of genome sequencing might be applicable to also inanimate/non-living things too! But that's a philosophical discussion for another day.

5 Conclusion

In this chapter, we have introduced the central concept of sequencing, and shown how it is applied to the genetics engineering problem of genome sequencing. We have formally specified the three core problems underlying genome sequencing as is currently explored in both literature and practice, and have especially given the matter a whole new

treatment using the mathematics of transformatics. We have demonstrated a detailed solution for the problem of computing the MCS, and have proposed solutions for the ORS and finally the computation of the genome sequence itself. We have specified a generic abstract machine capable of computing the genome sequence of any collection of sequences (not just genetic sequences), and have also shown how the idea of genome sequencing might be leveraged in practically relevant scenarios such as the evaluation of whether or not some two or more sequences belong to the same entity based on known or computed genome sequences — a problem that might even find immediate use in special kinds of artificial intelligence such as in genetic programming systems or perhaps in various kinds of bio-automata and even synthetic biology. Finally, a law, the Identity Genome Sequence law, has been proposed, and it is argued that it would find relevance beyond just genetics if well expounded upon with research and theory in other fields. This chapter is but one of many critical areas of genetics being studied, formalized and being furthered using the new mathematics of transformatics, and so, hopefully, the discussions and ideas kickstarted in this work shall spur the interest of those already familiar with what is going on in genetics, but also those coming to this work from related fields such computational biology, computer science, linear algebra, pure and applied mathematics, etc.

References

- [1] Joseph Willrich Lutalo. **TRANSFORMATICS 101 - explained**. *Thesis*, page 20, October 2025. <https://www.academia.edu/144344109/>.
- [2] Joseph Willrich Lutalo. **Applying TRANSFORMATICS in GENETICS**. *I*POW*, pages 1–206, Aug 2025. <https://www.academia.edu/143516392/>.
- [3] Valerie Illingworth, editor. *Oxford Dictionary of Computing*. Oxford University Press, Oxford, UK, 4 edition, 1996. Available at <https://openlibrary.org/books/OL412451M>.
- [4] Centers for Disease Control and Prevention. What is genomic sequencing? <https://www.cdc.gov/amd/whatis-genomic-sequencing.html>, 2024.
- [5] Microsoft Copilot. Clarifying discussions on dna sequence facts and genetics in general during drafting manuscript. AI-generated insights via Copilot discussion, 2026. Personal communication throughout preparation and revision of the manuscript.
- [6] Teresa Przytycka. Lecture 10: Whole genome sequencing and analysis. https://www.ncbi.xyz/CBBresearch/Przytycka/download/lectures/PCB_Lect10_Whole_Genome.pdf, 2025. Accessed August 2, 2025. Lecture slides from Introduction to Computational Biology.
- [7] Joseph Willrich Lutalo. **A general theory of number cardinality**. *Academia.edu*, Jan 2024. Accessible via https://www.academia.edu/43197243/A_General_Theory_of_Number_Cardinality.